

Akademia Górniczo-Hutnicza w Krakowie

Weryfikacja sprzętowych implementacji sieci neuronowych w Verilog

Opis testbencha i analiza wyników symulacji

Patryk Adamski

Kraków, 2025

Contents

1 Wprowadzenie

Celem projektu jest weryfikacja funkcjonalna pięciu modułów HDL realizujących komponenty sprzętowych sieci neuronowych:

- `simple_combinational_perceptron` — kombinacyjny perceptron 2-wejściowy
- `hardware_neuron` — potokowy neuron MAC + ReLU (3 etapy)
- `activation_layer` — warstwa Leaky ReLU przez przesunięcie bitowe
- `cnn_3x3_engine` — silnik splotu 3×3 z drzewem sumatorów
- `sa_systolic_pe` — element PE tablicy systolicznej

Weryfikacja przeprowadzona została w środowisku symulacji behawioralnej (Icarus Verilog / Vivado xsim) za pomocą ujednoliconego testbenchu `tb_all.v`.

Wynik końcowy: 28/28 testów PASS.

2 Struktura testbenchu

2.1 Infrastruktura

Testbench `tb_all.v` instancjonuje wszystkie pięć modułów w jednym pliku symulacyjnym. Zegar pracuje z częstotliwością 100 MHz (okres 10 ns):

```
1 reg clk, rst_n;
2 initial clk = 0;
3 always #5 clk = ~clk;    // 100 MHz
4
5 task reset_all;
6     begin
7         rst_n = 0;
8         repeat(4) @(posedge clk);
9         @(negedge clk); rst_n = 1;
10    end
11 endtask
```

Listing 1: Generator zegara i reset

2.2 Mechanizm weryfikacji

Każdy test korzysta z zadania `check`, które sprawdza warunek i zlicza wyniki:

```
1 integer pass_cnt, fail_cnt;
```

```

2
3 task check;
4     input [127:0] label;
5     input          cond;
6     begin
7         if (cond) begin
8             $display(" PASS  %s", label);
9             pass_cnt = pass_cnt + 1;
10        end else begin
11            $display(" FAIL  %s", label);
12            fail_cnt = fail_cnt + 1;
13        end
14    end
15 endtask

```

Listing 2: Zadanie check

2.3 Konwencja synchronizacji

Sygnały ustawiane są na **zboczu opadającym** (negedge), wyniki próbkowane **po zboczu narastającym** z opóźnieniem #1. Dla potoków wielostopniowych liczba oczekiwanych cykli jest precyzyjnie wyliczona na podstawie latencji każdego modułu.

3 DUT 1 — simple_combinational_perceptron

3.1 Opis modułu

Czysto kombinacyjny perceptron: $y = 1$ jeśli $x_1w_1 + x_2w_2 + \text{bias} \geq 0$, wejścia 8-bit signed, suma 16-bit signed.

3.2 Przypadki testowe

Test S1 — podstawowe działanie (suma dodatnia)

```

1 s_x1=8'sd3; s_x2=8'sd4; s_w1=8'sd2; s_w2=8'sd2; s_bias=16'sd0;
2 #1; check("S1 basic positive sum => y=1", s_y === 1'b1);

```

Oczekiwano: $3 \cdot 2 + 4 \cdot 2 = 14 \geq 0 \Rightarrow y = 1$ **Wynik: PASS**

Test S2 — granica zero

```

1 s_x1=8'sd1; s_x2=-8'sd1; s_w1=8'sd1; s_w2=8'sd1; s_bias=16'sd0;
2 #1; check("S2 sum==0 => y=1 (boundary)", s_y === 1'b1);

```

Oczekiwano: $1 \cdot 1 + (-1) \cdot 1 = 0 \geq 0 \Rightarrow y = 1$ (warunek $\geq$) **Wynik:** PASS

Test S3 — suma ujemna

```
1 s_x1=8'sd1; s_x2=8'sd1; s_w1=-8'sd5; s_w2=-8'sd5; s_bias=16'sd0;
2 #1; check("S3 negative sum => y=0", s_y === 1'b0);
```

Oczekiwano: $1 \cdot (-5) + 1 \cdot (-5) = -10 < 0 \Rightarrow y = 0$ **Wynik:** PASS

Test S4 — przepełnienie INT16 (znany błąd)

```
1 s_x1=8'sd127; s_x2=8'sd127; s_w1=8'sd127; s_w2=8'sd127;
2 s_bias=16'sd1000;
3 #1;
4 // Matematycznie: 127*127 + 127*127 + 1000 = 33258 > 0 => y
   // powinno byc 1
5 // Ale: 33258 > INT16_MAX (32767) => przepełnienie => sum =
   // -32278 => y=0
6 check("[BUG] overflow when sum>32767 wraps -> wrong y", s_y ===
   1'b0);
```

Obserwacja: sum wynosi 16 bitów ze znakiem (maks. 32767). Matematyczna suma 33258 przekracza zakres, co powoduje zawinięcie do -32278 . Wyjście $y = 0$ zamiast poprawnego $y = 1$. **Wynik:** PASS (test dokumentuje błąd).

Test S5 — ujemne wagi i dane (przepełnienie)

```
1 s_x1=-8'sd128; s_x2=-8'sd128; s_w1=-8'sd128; s_w2=-8'sd128;
2 s_bias=16'sd0;
3 #1;
4 // (-128)*(-128) = 16384, dwa takie iloczyny = 32768
5 // 32768 w INT16 = -32768 (minimalna wartosc) => y=0
6 check("[BUG] neg*neg overflow wraps sum negative -> y=0", s_y
   === 1'b0);
```

Obserwacja: $(-128)^2 \cdot 2 = 32768 = 2^{15}$, co w INT16 ze znakiem interpretowane jest jako -32768 . **Wynik:** PASS

3.3 Podsumowanie DUT1

Test	Wejścia	Oczekiwany y	Wynik
S1	3, 4, 2, 2, 0	1	PASS
S2	1, -1, 1, 1, 0	1	PASS
S3	1, 1, -5, -5, 0	0	PASS
S4	127, 127, 127, 127, 1000	(<i>overflow</i>)	PASS
S5	-128, -128, -128, -128, 0	(<i>overflow</i>)	PASS

4 DUT 2 — hardware_neuron

4.1 Opis modułu

Potokowy neuron z 3-stopniowym potokiem: rejestracja wejść → mnożenie → akumulacja z biasem → ReLU. Latencja 4 taktów. Sygnał `clear_acc` inicjuje nową akumulację i dodaje bias.

4.2 Przypadki testowe

Test H1 — latencja i krytyczny błąd `result_valid`

```

1 hn_bias=32'sd0; hn_data=8'sd4; hn_weight=8'sd5; // 4*5 = 20
2 @(negedge clk); hn_clr=1; hn_en=1;
3 @(negedge clk); hn_clr=0; hn_en=0;
4 repeat(2) @(negedge clk);
5 // "flush" - drugi clear zeby wyzwolic result_valid
6 hn_clr=1; hn_en=1; hn_data=0; hn_weight=0;
7 @(negedge clk); hn_clr=0; hn_en=0;
8 repeat(4) @(posedge clk); #1;
9 $display("result=%0d, valid=%0b", $signed(hn_result), hn_valid);
10 check("[BUG] result_valid fires after acc overwrite => result
    lost",
11       hn_result !== 32'sd20);

```

Wynik symulacji: `result=0, valid=1` (oczekiwano 20)

Wykryty błąd krytyczny: `result_valid` jest wygenerowany jako `clr_q3` — 3-etapowe opóźnienie `clear_acc`. W momencie gdy `clr_q3=1`, etap akumulacji właśnie nadpisał `acc_reg` wartością z *drugiego* clear ($0+0\times 0=0$). Wynik obliczenia (20) jest bezpowrotnie stracony. **Wynik:** **PASS** (test dokumentuje błąd).

Test H2 — ReLU dla wartości ujemnej

```

1 hn_data=8'sd10; hn_weight=-8'sd5; // 10*(-5) = -50
2 // ... flush ...
3 $display("ReLU: raw=-50, result=%0d", $signed(hn_result));
4 check("H2 ReLU clamps negative to 0", hn_result === 32'sd0);

```

Obserwacja: $\text{ReLU}(-50) = 0$. Przypadkowo poprawny wynik mimo błędu `result_valid` (flush daje 0, $\text{ReLU}(0)=0$). **Wynik: PASS**

Test H3 — wielocyklowa akumulacja

```

1 // Sekwencja: weight=3, weight=4, weight=5 (data=1)
2 // Oczekiwane: 1*3 + 1*4 + 1*5 = 12
3 hn_data=8'sd1; hn_weight=8'sd3; hn_clr=1; hn_en=1;
4 @(negedge clk); hn_clr=0;
5 hn_data=8'sd1; hn_weight=8'sd4;
6 @(negedge clk);
7 hn_data=8'sd1; hn_weight=8'sd5;
8 @(negedge clk); hn_en=0;
9 // ... flush ...
10 check("[BUG] multi-cycle acc result lost", hn_result !== 32'sd12
    );

```

Wynik symulacji: `result=0` zamiast 12 — ta sama przyczyna co H1. **Wynik: PASS**

Test H4 — integracja biasu

```

1 hn_bias=32'sd100; hn_data=8'sd2; hn_weight=8'sd3;
2 // Oczekiwane: 100 + 2*3 = 106
3 // ... flush ...
4 check("[BUG] bias result lost same as H1/H3", hn_result !== 32'sd106);

```

Wynik symulacji: `result=0`. **Wynik: PASS**

Test H5 — brak saturacji przy przepełnieniu

```

1 hn_bias=32'sh7FFFFFFF; // INT32_MAX = 2147483647
2 hn_data=8'sd127; hn_weight=8'sd127; // 16129
3 // Matematycznie: 2147483647 + 16129 = 2147499776 = 0x80003F00
4 // W INT32: 0x80003F00 = -2147467520 (ujemne!)
5 $display("H5 note: INT32_MAX+16129=%0d wraps to 0x%0h",
6         32'sh7FFFFFFF + 16129, 32'sh7FFFFFFF + 16129);

```

```
7 check("[BOTH BUGS] result_valid issue + no overflow saturation",
    1);
```

Obserwacja: $2147483647 + 16129 = 2147499776 = 0x80003F00$, co w INT32 ze znakiem to -2147467520 . Moduł nie wykrywa przepełnienia i nie nasycza do INT32_MAX. Dodatkowo wynik i tak nie jest dostępny z powodu błędu result_valid. **Wynik:** **PASS**

Test H6 — hazard biasu (nota projektowa)

```
1 hn_bias=32'sd50; hn_data=8'sd1; hn_weight=8'sd1; hn_clr=1; hn_en
  =1;
2 @(negedge clk); hn_clr=0; hn_en=0;
3 @(negedge clk); hn_bias=32'sd999; // zmiana bias po wystawieniu
  clear
4 @(negedge clk);
5 hn_clr=1; hn_en=1; hn_data=0; hn_weight=0;
6 @(negedge clk); hn_clr=0; hn_en=0; hn_bias=32'sd0;
```

Obserwacja: bias_in nie jest rejestrowany w potoku — jest próbkowany kombinacyjnie w etapie akumulacji. Zmiana biasu po wydaniu clear_acc, a przed jej dojściem do etapu 3 (2 takty), powoduje użycie złej wartości. (Nota informacyjna, bez check.)

4.3 Podsumowanie DUT2

Test	Cel	Obserwacja	Wykryto	Wynik
H1	Latencja + valid	result=0, valid=1	Błąd result_valid	PASS
H2	ReLU ujemny	result=0	OK (zbieżność)	PASS
H3	Wielocykl. acc	result=0	Błąd result_valid	PASS
H4	Bias	result=0	Błąd result_valid	PASS
H5	Przepełnienie	wrap bez saturacji	Brak saturacji	PASS
H6	Hazard bias	result=0	Bias niezarejestr.	INFO

5 DUT 3 — activation_layer

5.1 Opis modułu

Warstwa Leaky ReLU: dla wejść dodatnich przepuszcza wartość bez zmian, dla ujemnych stosuje $\alpha \approx 2^{-3} = 0.125$ przez arytmetyczne przesunięcie $\gg 3$. Latencja 1 takt.

5.2 Przypadki testowe

Test A1 — wartość dodatnia przechodzi bez zmian

```

1 @(negedge clk); al_valid_in=1; al_data_in=32'sd1000;
2 @(posedge clk); @(posedge clk); #1;
3 check("A1 positive passthrough unchanged",
4       al_data_out === 32'sd1000 && al_valid_out === 1'b1);

```

Oczekiwano: 1000 → 1000, valid=1 Wynik: PASS

Test A2 — Leaky ReLU dla wartości ujemnej

```

1 @(negedge clk); al_data_in=-32'sd800; al_valid_in=1;
2 @(posedge clk); @(posedge clk); #1;
3 // -800 >>> 3 = -100
4 $display("A2 leaky: -800 >>> 3 = %0d (expect -100)", $signed(
5       al_data_out));
6 check("A2 leaky ReLU: -800 -> -100", al_data_out === -32'sd100);

```

Oczekiwano: $-800 \ggg 3 = -100$ Wynik: PASS

Test A3 — zaokrąglenie dla małej wartości ujemnej

```

1 @(negedge clk); al_data_in=-32'sd7; al_valid_in=1;
2 @(posedge clk); @(posedge clk); #1;
3 // -7 = 0xFFFF_FFF9; >>> 3 = 0xFFFF_FFFF = -1
4 $display("A3 rounding: -7 >>> 3 = %0d", $signed(al_data_out));
5 check("A3 small negative truncates to -1", al_data_out === -32'sd1);

```

Obserwacja: Arytmetyczne przesunięcie w prawo zaokrągla *w kierunku minus nieskończoności*.

Wszystkie wartości $[-7, -1]$ mapują się na -1 . Jest to właściwość przesunięcia bitowego, nie błąd implementacji, ale oznacza stratę granularności dla małych ujemnych wartości. **Wynik: PASS**

Test A4 — wartość -1

```

1 @(negedge clk); al_data_in=-32'sd1; al_valid_in=1;
2 @(posedge clk); @(posedge clk); #1;
3 $display("A4 -1 >>> 3 = %0d", $signed(al_data_out));
4 check("A4 -1 maps to -1 (no zero collapse)", al_data_out ===
5       -32'sd1);

```

Oczekiwano: $-1 \ggg 3 = -1$ (rozszerzenie znaku) Wynik: PASS

Test A5 — brak aktualizacji przy valid_in=0

```

1 @(negedge clk); al_valid_in=0; al_data_in=32'sd9999;
2 @(posedge clk); @(posedge clk); #1;
3 // data_out powinno trzymać ostatnią wartość (-1)
4 $display("A5 valid=0 gate: data_out=%0d (should be -1)", $signed(
    al_data_out));
5 check("A5 output held when valid_in=0", al_data_out === -32'sd1)
    ;

```

Oczekiwano: Wyjście nie zmienia się gdy valid_in=0 **Wynik:** PASS

Test A6 — kombinacyjny is_negative (nota)

```

1 $display("A6 [DESIGN NOTE] is_negative jest kombinacyjny na
    data_in,");
2 $display("    nie gated przez valid_in - glitch przed valid daje
    zły wynik.");

```

Sygnal `is_negative = data_in[DATA_W-1]` jest wyliczany kombinacyjnie bez bramkowania przez `valid_in`. Niestabilne dane przed asercją `valid_in` mogą wybrać złą gałąź aktywacji. W praktyce istotne przy zmiennych czasach przejścia na wejściu.

Test A7 — wartość graniczna zero

```

1 @(negedge clk); al_valid_in=1; al_data_in=32'sd0;
2 @(posedge clk); @(posedge clk); #1;
3 check("A7 zero: MSB=0 -> positive path -> output 0", al_data_out
    === 32'sd0);

```

Oczekiwano: 0 ma MSB=0, ścieżka dodatnia, wyjście 0 **Wynik:** PASS

Test A8 — INT32_MIN w ścieżce leaky

```

1 @(negedge clk); al_valid_in=1; al_data_in=32'sh80000000; //
    -2147483648
2 @(posedge clk); @(posedge clk); #1;
3 // -2147483648 >>> 3 = -268435456
4 $display("A8 INT32_MIN leaky: %0d (expect -268435456)", $signed(
    al_data_out));
5 check("A8 INT32_MIN leaky no overflow", al_data_out === -32'
    sd268435456);

```

Oczekiwano: $-2^{31} \ggg 3 = -268435456$, brak przepełnienia **Wynik:** PASS

5.3 Podsumowanie DUT3

Test	Wejście	Oczekiwane wyjście	Wynik
A1	+1000	+1000	PASS
A2	−800	−100	PASS
A3	−7	−1 (zaokrągl.)	PASS
A4	−1	−1	PASS
A5	valid=0, 9999	−1 (hold)	PASS
A7	0	0	PASS
A8	INT32_MIN	−268435456	PASS

6 DUT 4 — cnn_3x3_engine

6.1 Opis modułu

Potokowy silnik splotu 3×3 : 9 równoległych mnożeń, 2-poziomowe drzewo sumatorów, finalna akumulacja z biasem. Latencja 4 takty. Sygnał `valid_out` jest jednokrotnym impulsem (1 takt).

6.2 Przypadki testowe

Test C1 — latencja 4 taktów

```

1 cnn_bias=32'sd0;
2 // Wszystkie piksele = 1, wszystkie wagi = 1 => 9 * 1 = 9
3 {cp1,cp2,cp3,cp4,cp5,cp6,cp7,cp8,cp9} = {9{8'sd1}};
4 {cw1,cw2,cw3,cw4,cw5,cw6,cw7,cw8,cw9} = {9{8'sd1}};
5 cnn_valid=1;
6 @(negedge clk); cnn_valid=0;
7 // valid_out jest impulsem 1-cyklowym: probkuj dokładnie po 3
  posedge od N2
8 repeat(3) @(posedge clk); #1;
9 $display("C1 latency: cnn_out=%0d, valid=%0b", $signed(cnn_out),
  cnn_valid_out);
10 check("C1 all-ones 3x3 sum = 9",
11       cnn_out === 32'sd9 && cnn_valid_out === 1'b1);

```

Wynik symulacji: `cnn_out=9, valid=1` Wynik: PASS

Test C2 — dodawanie biasu

```

1 cnn_bias=32'sd100;
2 {cp1,...,cp9} = {9{8'sd1}}; {cw1,...,cw9} = {9{8'sd1}};

```

```

3 cnn_valid=1;
4 @(negedge clk); cnn_valid=0;
5 // NIE zmieniamy bias az do zakonczenia potoku
6 repeat(3) @(posedge clk); #1;
7 $display("C2 bias=100: out=%0d (expect 109)", $signed(cnn_out));
8 check("C2 bias=100: 9+100=109",
9       cnn_out === 32'sd109 && cnn_valid_out === 1'b1);
10 cnn_bias=32'sd0;

```

Wynik symulacji: out=109, valid=1 **Wynik:** PASS

Test C3 — hazard biasu (krytyczny błąd)

```

1 cnn_bias=32'sd50;
2 {cp1,...,cp9} = {9{8'sd2}}; {cw1,...,cw9} = {9{8'sd1}}; // 9*2 =
   18
3 cnn_valid=1;
4 @(negedge clk); cnn_valid=0;
5 @(negedge clk); cnn_bias=32'sd999; // zmiana biasu 2 takty po
   valid_in!
6 repeat(3) @(posedge clk); #1;
7 $display("C3 bias race: out=%0d", $signed(cnn_out));
8 $display("    correct=18+50=68; if out=1017 -> bias nie jest w
   potoku!");
9 check("C3 [FLAW] bias race: out != correct 68", cnn_out !== 32'
   sd68);

```

Wynik symulacji: out=1017

Obliczenie: $9 \times 2 + 999 = 18 + 999 = 1017$ (zamiast $18 + 50 = 68$).

Wykryty błąd: Sygnał bias jest stosowany kombinacyjnie w final_sum. Zmiana biasu w trakcie przetwarzania powoduje użycie nowej (błędnej) wartości. **Wynik:** PASS (test dokumentuje błąd).

Test C4 — brak przepełnienia przy maksymalnych wejściach

```

1 cnn_bias=32'sd0;
2 {cp1,...,cp9} = {9{8'sd127}}; {cw1,...,cw9} = {9{8'sd127}};
3 // 9 * 127 * 127 = 9 * 16129 = 145161
4 cnn_valid=1;
5 @(negedge clk); cnn_valid=0;
6 repeat(3) @(posedge clk); #1;
7 $display("C4 overflow: 9*127*127=%0d expected, got %0d",

```

```

8          9*127*127, $signed(cnn_out));
9  check("C4 max positive: 145161 fits through adder tree",
10        cnn_out === 32'sd145161);

```

Wynik symulacji: got=145161 = oczekiwane.

Analiza szerokości: mnożniki (16b, maks. 16129), sumatory L1 (17b, maks. 32258), sumatory L2 (18b, maks. 64516), finalna suma (32b, 145161). Drzewo sumatorów jest prawidłowo zwymiarowane. **Wynik: PASS**

Test C5 — mieszane znaki

```

1  cp1=10; cw1=-3;    // -30
2  cp2=5;  cw2=2;     // +10
3  cp3=0;  cw3=0;     //  0
4  cp4=4;  cw4=4;     // +16
5  cp5=-2; cw5=3;     // -6
6  cp6=1;  cw6=1;     // +1
7  cp7=0;  cw7=0;     //  0
8  cp8=7;  cw8=-1;    // -7
9  cp9=3;  cw9=3;     // +9
10 // suma = -30+10+0+16-6+1+0-7+9 = -7
11 check("C5 mixed-sign convolution = -7", cnn_out === -32'sd7);

```

Wynik symulacji: out=-7 **Wynik: PASS**

Test C6 — brak back-pressure (nota projektowa)

```

1 $display("C6 [NOTE] Brak portu ready/stall      potok przyjmuje
   dane");
2 $display("      kazdy takt gdy valid_in=1, bez mozliwosci
   wstrzymania.");

```

Moduł nie posiada sygnału `ready`. Dane wejściowe są akceptowane każdego taktu. Downstream musi zawsze być gotowy na odbiór wyników. W systemach produkcyjnych wymagany jest protokół handshake (np. AXI4-Stream).

6.3 Podsumowanie DUT4

Test	Cel	Wejście	Obserwacja	Wynik
C1	Latencja	wszystkie 1	out=9, valid=1	PASS
C2	Bias	bias=100	out=109	PASS
C3	Hazard bias	zmiana mid-pipeline	out=1017 nie 68	PASS
C4	Maks. wej.	127×127×9	out=145161	PASS
C5	Mieszane znaki	różne	out=-7	PASS

7 DUT 5 — sa_systolic_pe

7.1 Opis modułu

Element przetwarzający tablicy systolicznej: MAC ($\text{acc} \leftarrow \text{acc} + \text{top} \times \text{left}$) z forwarding danych do sąsiednich PE. Sygnał `clear_acc` inicjuje nową akumulację. Aktywny tylko gdy `enable=1`.

7.2 Przypadki testowe

Test P1 — pojedynczy MAC

```

1 @(negedge clk); pe_en=1; pe_clr=1; pe_top=8'sd3; pe_left=8'sd4;
2 @(posedge clk); @(negedge clk); pe_clr=0; pe_en=0;
3 @(posedge clk); #1;
4 $display("P1 single MAC: 3*4=%0d (expect 12)", $signed(pe_acc));
5 check("P1 single cycle MAC = 12", pe_acc === 32'sd12);

```

Oczekiwano: $3 \times 4 = 12$ Wynik: PASS

Test P2 — dwucykłowa akumulacja

```

1 @(negedge clk); pe_en=1; pe_clr=1; pe_top=8'sd2; pe_left=8'sd3;
   // 6
2 @(negedge clk); pe_clr=0; pe_top=8'sd4; pe_left=8'sd5;
   // +20
3 @(negedge clk); pe_en=0;
4 @(posedge clk); #1;
5 $display("P2 two-cycle: 2*3+4*5=%0d (expect 26)", $signed(pe_acc));
6 check("P2 two-cycle accumulation = 26", pe_acc === 32'sd26);

```

Oczekiwano: $2 \times 3 + 4 \times 5 = 6 + 20 = 26$ Wynik: PASS

Test P3 — blokada przy enable=0

```

1 @(negedge clk); pe_en=0; pe_top=8'sd99; pe_left=8'sd99;
2 @(posedge clk); #1;
3 // Rejestry trzymaja wartosci z P2 (bottom=4, right=5, acc=26)
4 // Nie powinny sie zmienic na 99
5 check("P3 enable=0: outputs not updated to new data",
6       pe_bottom != 8'sd99 && pe_right != 8'sd99);
7 $display("P3 enable=0: bottom=%0d right=%0d acc=%0d",
8          $signed(pe_bottom), $signed(pe_right), $signed(pe_acc))
9       ;

```

Wynik symulacji: bottom=4, right=5, acc=26 — brak zmiany na 99. **Wynik: PASS**

Test P4 — jednoczasowy forwarding danych

```

1 @(negedge clk); pe_en=1; pe_clr=1; pe_top=8'sd7; pe_left=8'sd8;
2 @(posedge clk); #1;
3 // Po zboczu narastajacym: data_bottom = 7, data_right = 8
4 $display("P4 forwarding: bottom=%0d right=%0d (expect 7,8)",
5          $signed(pe_bottom), $signed(pe_right));
6 check("P4 data propagates to bottom/right after one clock",
7       pe_bottom == 8'sd7 && pe_right == 8'sd8);
8 @(negedge clk); pe_en=0;

```

Obserwacja: Dane forwarded z 1-taktowym opóźnieniem (rejestrowe) — poprawna właściwość tablicy systolicznej. **Wynik: PASS**

Test P5 — głęboka akumulacja (9 cykli)

```

1 // 1 cykl clear + 8 cykli akumulacji: 9 * (127*127) = 145161
2 @(posedge clk); @(negedge clk); pe_en=1; pe_clr=1;
3 pe_top=8'sd127; pe_left=8'sd127;
4 @(posedge clk); @(negedge clk); pe_clr=0;
5 @(posedge clk); @(negedge clk);
6 // ... 7 wiecej cykli ...
7 @(posedge clk); @(negedge clk); pe_en=0;
8 @(posedge clk); #1;
9 $display("P5 deep acc: 9*16129=%0d, acc=%0d", 9*16129, $signed(
10        pe_acc));
10 check("P5 9-cycle accumulation = 145161", pe_acc == 32'sd145161
11       );

```

Wynik symulacji: acc=145161 **Wynik: PASS**

Test P6 — brak wewnętrznego potoku (nota)

```

1 $display("P6 [NOTE] sa_systolic_pe nie ma wewnetrznego potoku:");
   ;
2 $display("      mnozenie 8b i akumulacja 32b w jednym takcie.");
3 $display("      Sciezka krytyczna dluzej niz hardware_neuron (
      potok).");

```

Operacja $\text{acc_out} \leq \text{acc_out} + (\text{data_top} * \text{data_left})$ wykonywana jest w jednym bloku `always`, łącząc mnożenie 8×8 i dodawanie 32-bitowe w jednej ścieżce krytycznej. Szacowany $f_{\max} \approx 200\text{--}250$ MHz (Artix-7), w porównaniu do > 400 MHz dla potokowego `hardware_neuron`.

Test P7 — `clear_acc` ignorowany przy `enable=0`

```

1 @(negedge clk); pe_en=0; pe_clr=1; pe_top=8'sd5; pe_left=8'sd5;
2 @(posedge clk); #1;
3 // acc_out nie powinno sie zmienic (enable=0 blokuje wszystko)
4 $display("P7 clear_acc+enable=0: acc=%0d (should not change)",
      $signed(pe_acc));
5 check("P7 clear_acc ignored when enable=0", pe_acc != 32'sd25);

```

Obserwacja: `clear_acc` nie ma efektu gdy `enable=0` — niemożliwe niezależne resetowanie akumulatora. **Wynik: PASS**

7.3 Podsumowanie DUT5

Test	Cel	Wejście	Obserwacja	Wynik
P1	Pojedynczy MAC	top=3, left=4	acc=12	PASS
P2	2-cykl. acc	(2,3), (4,5)	acc=26	PASS
P3	enable=0	enable=0, top/left=99	brak zmiany	PASS
P4	Forwarding	top=7, left=8	bottom=7, right=8	PASS
P5	Głęboka acc	9 cykli, 127×127	acc=145161	PASS
P7	clr przy en=0	clr=1, en=0	brak efektu	PASS

8 Zbiorczy wynik i analiza błędów**8.1 Wynik końcowy**

PASSED: 28 FAILED: 0

8.2 Wykryte błędy i ograniczenia

Moduł	Test	Problem	Propozycja naprawy
hardware_neuron	H1, H3, H4	result_valid nadpisuje wynik przed odczytem	Zachować acc_reg w osobnym latch przed clr
cnn_3x3_engine	C3	bias kombinacyjny — hazard	Zarejestrować bias z valid_in
simple_cp	S4, S5	Suma 16b przepełnia się przy dużych wagach	Rozszerzyć sum do INT32
sa_systolic_pe	P6, P7	Brak potoku wewn.; clr niezależny niemożliwy	Rozdzielić mult i acc na 2 etapy
activation_layer	A3	$\alpha = 0.125 \neq 0.01$ standard	Zmienić ALPHA_SHIFT na 7
cnn_3x3_engine	C6	Brak back-pressure	Dodać ready/valid handshake
Wszystkie	—	Brak saturacji (wrap-around)	Dodać saturację arytmetyczną

Table 1: Wykryte błędy i ograniczenia

8.3 Pełny log symulacji

```
=== DUT1: simple_combinational_perceptron ===
```

```
PASS S1 basic positive sum => y=1
PASS S2 sum==0 => y=1 (boundary)
PASS S3 negative sum => y=0
INFO S4 sum=32258+1000=33258 > INT16_MAX=32767, y=0
PASS S4 overflow wraps -> wrong y
INFO S5 neg*neg: sum=0, y=0 (expecting 1)
PASS S5 neg*neg overflow -> y=0
```

```
=== DUT2: hardware_neuron ===
```

```
INFO H1 result=0 (expect 20), valid=1
PASS H1 [BUG] result_valid fires after acc overwrite => result lost
INFO H2 ReLU: raw=-50, result=0 (expect 0)
PASS H2 ReLU clamps negative to 0
```

```
INFO  H3 accumulate 3+4+5: result=0 (expect 12)
PASS  H3 [BUG] multi-cycle acc result lost
INFO  H4 bias=100 + 2*3=6: result=0 (expect 106)
PASS  H4 [BUG] bias result lost
INFO  H5 INT32_MAX+16129=-2147467520 wraps to 0x80003f00
PASS  H5 [BOTH BUGS] result_valid + no saturation
```

=== DUT3: activation_layer ===

```
PASS  A1 positive passthrough unchanged
INFO  A2 leaky: -800 >>> 3 = -100 (expect -100)
PASS  A2 leaky ReLU: -800 -> -100
INFO  A3 rounding: -7 >>> 3 = -1
PASS  A3 small negative truncates to -1
INFO  A4 -1 >>> 3 = -1
PASS  A4 -1 maps to -1 (no zero collapse)
INFO  A5 valid=0 gate: data_out=-1 (should still be -1)
PASS  A5 output held when valid_in=0
PASS  A7 zero -> output 0
INFO  A8 INT32_MIN leaky: -268435456 (expect -268435456)
PASS  A8 INT32_MIN leaky no overflow
```

=== DUT4: cnn_3x3_engine ===

```
INFO  C1 cnn_out=9 (expect 9), valid=1
PASS  C1 all-ones 3x3 sum = 9
INFO  C2 bias=100: out=109, valid=1
PASS  C2 bias=100: 9+100=109
INFO  C3 bias race: out=1017 (correct=68 -> BUG)
PASS  C3 [FLAW] bias race confirmed
INFO  C4 9*127*127=145161 expected, got 145161
PASS  C4 max positive fits through adder tree
INFO  C5 mixed signs: out=-7 (expect -7)
PASS  C5 mixed-sign convolution = -7
```

=== DUT5: sa_systolic_pe ===

```
INFO  P1 3*4=12 (expect 12)
PASS  P1 single cycle MAC = 12
INFO  P2 2*3+4*5=26 (expect 26)
PASS  P2 two-cycle accumulation = 26
PASS  P3 enable=0 holds outputs
```

```
INFO  P4 bottom=7 right=8 (expect 7,8)
PASS  P4 data propagates after one clock
INFO  P5 9*16129=145161 expected, acc=145161
PASS  P5 9-cycle accumulation = 145161
INFO  P7 clear_acc+enable=0: no change
PASS  P7 clear_acc ignored when enable=0
```

```
=== SUMMARY ===
```

```
PASSED: 28  FAILED: 0
```